
Universidad Nacional Autónoma de México

Facultad de Estudios Superiores Acatlán

Diplomado en Ciencia de Datos

Generación 33

Proyecto Final Nexus Sentinel

*Detección de uso indebido de credenciales mediante analítica de
comportamiento de usuarios y entidades (UEBA)*

Presentado a: Fernando Barranco Rodríguez

Autor: Arellano Ortiz Andre

Fecha: 2 de agosto del 2026

Ciudad de México, México

Índice

1. Introducción	3
1.1. Empresa y contexto de negocio	3
1.2. Flujo de trabajo	4
2. Planteamiento del problema	4
2.1. Decisiones de diseño no negociables	5
3. Estrategia de negocio	5
3.1. Nicho de negocio	5
3.2. Usuarios finales y accionables	6
4. Descripción de los datos	6
4.1. Primer conflicto: la adquisición de los datos	7
4.2. Muestreo justificado	7
4.3. Calidad de los datos	8
5. Análisis exploratorio de datos	8
5.1. El ritmo de la red	8
5.2. Las señales del compromiso	9
5.3. El movimiento lateral hecho visible	9
5.4. El desbalance	11
6. Ingeniería de variables	11
6.1. La regla anti-fuga	12
6.2. Etiquetado y partición temporal	12
7. Modelado	12
7.1. Los tres conjuntos	13
7.2. Modelos no supervisados	13
7.3. Segundo conflicto: el agrupamiento degenerado	13
7.4. Modelo supervisado central	14
7.5. Iteración de variables	14
7.6. Selección del mejor modelo y calibración	14
7.7. Evaluación final fuera de tiempo	15
7.8. Reporte de estabilidad	15
7.9. Explicabilidad	16
8. Análisis crítico y conflictos del desarrollo	17
8.1. Tercer conflicto: una autocorrección metodológica	17

8.2.	La exploración honesta de mejoras	18
8.3.	Contextualización honesta de los resultados	18
9.	Resolución del problema	18
9.1.	Pipeline de inferencia y puntuación de riesgo	18
9.2.	Consumo de modelos: la API	19
9.3.	La interfaz de usuario	19
9.4.	Asistente conversacional	19
10.	Conclusiones	20
11.	Referencias	20

1. Introducción

El perímetro de seguridad tradicional —firewall, antivirus, sistemas de detección de intrusiones— fue diseñado para detener a un intruso que *rompe* algo. Sin embargo, la mayoría de las brechas modernas no rompen nada: utilizan **credenciales legítimas**, robadas mediante *phishing*, compradas en mercados ilícitos o abusadas por un empleado, para moverse dentro de la red como si fueran un usuario normal. Este es el problema que la industria denomina UEBA (*User and Entity Behavior Analytics*) y que, en la práctica, resulta indistinguible de la amenaza interna: en ambos casos el atacante opera dentro de canales autorizados y cada autenticación individual es, técnicamente, válida.

En este proyecto desarrollé **Nexus Sentinel**, un sistema de puntuación de riesgo que detecta este tipo de comportamiento sobre datos reales de autenticación de una red corporativa. Concebí el proyecto en torno a una premisa que gobernó todas mis decisiones de diseño: *el modelo no debe ver eventos, sino desviaciones*. Una autenticación aislada no dice nada; lo que delata a una cuenta comprometida es la desviación de su patrón de comportamiento habitual. A lo largo del documento narro, en primera persona, el desarrollo completo: desde la adquisición de mil millones de eventos crudos hasta el despliegue de una interfaz funcional en producción, pasando por la ingeniería de variables —que considero el corazón técnico del trabajo— y una evaluación que me esforcé por mantener honesta incluso cuando los resultados no fueron los que esperaba.

La tarea que abordo es de **clasificación binaria supervisada** sobre la unidad de análisis **usuario-día**: dado el resumen del comportamiento de autenticación de un usuario durante un día, el sistema debe estimar la probabilidad de que ese día contenga actividad de compromiso. El problema presenta los retos estructurales que definen a la ciencia de datos aplicada a la ciberseguridad: un desbalance de clases extremo (menos del 0.4 % de los casos son positivos), un volumen masivo de datos (más de mil millones de eventos), datos desidentificados por privacidad y la necesidad de que el resultado sea *explicable*, pues ninguna acción sobre una cuenta o una persona puede tomarse a ciegas.

1.1 Empresa y contexto de negocio

Enmarqué el proyecto en **Nexus Sentinel**, una empresa ficticia de tecnología de tamaño medio que sufrió un incidente representativo del problema: la cuenta de uno de sus ingenieros fue utilizada durante semanas para explorar servidores a los que ese ingeniero jamás accedía. Ninguna alerta se disparó, porque cada autenticación individual era «válida». El incidente se detectó tarde y de forma casi accidental. Este caso ficticio —que, como mostraré en la presentación, tiene equivalentes reales y costosos

en la industria— define con precisión el problema que mi sistema busca resolver: no vigilar eventos individuales, sino la **desviación del patrón de comportamiento**.

1.2 Flujo de trabajo

El desarrollo que documento comprende ocho fases: (1) adquisición y muestreo justificado de los datos crudos; (2) análisis exploratorio orientado a descubrir empíricamente las señales de compromiso; (3) ingeniería de variables en cuatro niveles de desviación; (4) modelado supervisado y no supervisado con evaluación y análisis de estabilidad; (5) construcción del *pipeline* de inferencia y la puntuación de riesgo; (6) exposición de los modelos mediante una API; (7) desarrollo de una interfaz web con un asistente conversacional; y (8) la redacción de este documento. Cada fase la cerré con una verificación explícita, y varias de ellas revelaron conflictos técnicos y metodológicos que documento con transparencia, pues considero que la forma en que los resolví es parte sustancial del valor del trabajo.

2. Planteamiento del problema

La analítica de comportamiento de usuarios y entidades parte de una observación empírica bien documentada: el uso de credenciales robadas o abusadas es uno de los vectores más frecuentes en las brechas de seguridad modernas. El *Data Breach Investigations Report* de Verizon reporta año con año que el elemento humano y el uso de credenciales comprometidas participan en una fracción mayoritaria de los incidentes analizados [1]. La razón es estructural: robar una credencial es más barato y más silencioso que explotar una vulnerabilidad técnica, y una vez dentro, el atacante hereda todos los permisos legítimos de la cuenta.

El problema técnico central es que las herramientas perimetrales clásicas —SIEM, EDR, firewalls— razonan a nivel de *evento*: una regla se dispara cuando ocurre algo explícitamente prohibido. Pero el uso indebido de credenciales no viola ninguna regla a nivel de evento: la cuenta tiene permiso para autenticarse. La señal solo existe a nivel de *patrón*: un usuario que de pronto se autentica contra decenas de máquinas que jamás había tocado, en horarios inusuales o mediante protocolos que normalmente no emplea. Detectar esto requiere construir, para cada usuario, una línea base de comportamiento y medir cuánto se desvía de ella cada día.

2.1 Decisiones de diseño no negociables

Antes de escribir una sola línea de código fijé cinco decisiones de diseño que mantuve durante todo el proyecto, precisamente porque son las que separan una solución operativa de un ejercicio académico:

1. **Unidad de análisis = usuario-día.** El riesgo se evalúa sobre el resumen diario del comportamiento de cada usuario, no sobre eventos aislados.
2. **Métricas para desbalance extremo.** Empleo PR-AUC como métrica rectora, recall a una tasa de falsos positivos fija y F1 al umbral operativo. Nunca *accuracy*: con menos del 0.4 % de positivos, un clasificador que prediga siempre «normal» acierta el 99.6 % de las veces siendo completamente inútil. La literatura respalda esta elección: la curva de precisión-recall es más informativa que la ROC al evaluar clasificadores binarios sobre conjuntos desbalanceados [9].
3. **Humano en el circuito.** El sistema prioriza y explica; ninguna acción sobre una cuenta o una persona es automática. La decisión final siempre es de un analista.
4. **Umbrales calibrados por capacidad operativa.** Los umbrales de alerta se fijan por percentiles que producen un número de tickets diarios revisable por el equipo de seguridad, no mediante números arbitrarios.
5. **Muestreo documentado, no oculto.** El volumen masivo se gestiona con un muestreo justificado y trazable, jamás escondiendo la reducción de datos.

3. Estrategia de negocio

3.1 Nicho de negocio

Posicioné a Nexus Sentinel como una capa de **UEBA / Human Risk Analytics**: inteligencia interna para el área de ciberseguridad. La propuesta no compite con el SIEM ni con el EDR, sino que consume sus registros y devuelve una priorización explicada. La propuesta de valor es medible: reducir el volumen de alertas que un analista debe revisar manualmente, manteniendo el *recall* sobre los compromisos reales. En otras palabras, convertir un pajar de mil millones de eventos en una lista corta, priorizada y justificada de casos investigables.

3.2 Usuarios finales y accionables

Diseñé el sistema para tres perfiles de usuario, con el analista de un Centro de Operaciones de Seguridad (SOC) como usuario principal:

- **Analista SOC (principal):** necesita una cola priorizada y explicada, no un mar de alertas. El sistema le entrega el triaje por riesgo, la explicación de cada caso y las acciones de decisión.
- **CISO (secundario):** necesita la tendencia de riesgo organizacional y evidencia para justificar inversión. Le entrega un panel ejecutivo con el riesgo agregado y la eficiencia del SOC.
- **Oficial de Cumplimiento (implícito):** necesita auditabilidad de las decisiones sobre cuentas. Le entrega la trazabilidad completa desde las variables hasta la decisión humana registrada.

Traduje la puntuación de riesgo (0–100) a tres niveles accionables mediante un semáforo, cuya correspondencia con las acciones del SOC es uno a uno: **verde** (registro pasivo que alimenta la línea base del usuario), **naranja** (ticket con la evidencia y su explicación, para revisión) y **rojo** (investigación prioritaria con validación humana obligatoria antes de cualquier acción sobre la cuenta).

4. Descripción de los datos

Trabajé con el conjunto de datos *Comprehensive, Multi-Source Cyber-Security Events*, publicado por el Laboratorio Nacional de Los Álamos (LANL) y liberado por su autor para investigación pública [2, 3]. Es un conjunto de datos **reales**: 58 días consecutivos de eventos desidentificados de la red corporativa interna de LANL, con eventos de un equipo de *red team* (ataques reales controlados) que sirven como *ground truth* de comportamiento malicioso. En total, el conjunto documenta 1 648 275 307 eventos de 12 425 usuarios y 17 684 computadoras.

Utilicé dos de sus archivos: `auth.txt.gz` (7.2 GB comprimidos, aproximadamente 1 051 millones de autenticaciones Windows/Active Directory), que es la fuente principal, y `redteam.txt.gz` (4.8 KB), que contiene los eventos de compromiso que uso como etiquetas. Elegí esta fuente precisamente porque su desbalance extremo, su volumen masivo y su desidentificación reproducen las condiciones exactas que un científico de datos enfrenta en un SOC real. Cabe destacar que la guía de la rúbrica veta explícitamente a Kaggle como fuente de datos; LANL es una fuente institucional primaria y de acceso directo, lo que la hace idónea.

4.1 Primer conflicto: la adquisición de los datos

El primer obstáculo real del proyecto apareció en la adquisición. La documentación y los trabajos previos sobre este conjunto asumen una descarga directa y anónima desde el portal de LANL. Sin embargo, al momento de desarrollar el proyecto el portal había incorporado un *data fence*: un formulario que exige un correo electrónico y una declaración de uso, y que a cambio devuelve un *token* temporal con el que se construye la URL de descarga. Tuve que adaptar mi módulo de adquisición para solicitar ese token programáticamente antes de poder descargar los archivos, y documenté la fecha, la URL y el tamaño de cada descarga para garantizar la reproducibilidad.

El segundo obstáculo fue el volumen. Descargar 7.2 GB comprimidos y, sobre todo, procesarlos, exigía una estrategia cuidadosa: implementé la descarga en *streaming* con reanudación automática —que efectivamente tuvo que reanudarse tras una interrupción a mitad de la transferencia— y la lectura del archivo comprimido por bloques (*chunks*), sin descomprimirlo nunca por completo a disco. Esta decisión de ingeniería fue la que hizo viable el resto del proyecto en un equipo de cómputo personal.

4.2 Muestreo justificado

Mil millones de eventos ni caben cómodamente en memoria ni son necesarios. Gestioné el volumen con un muestreo que documenté por completo, en lugar de ocultarlo. Las reglas y su justificación fueron:

- **Ventana temporal:** los primeros 30 días. Verifiqué empíricamente sobre el archivo completo de 58 días que el 100 % de los eventos del *red team* ocurre entre los días 1 y 29, de modo que la ventana conserva la totalidad del *ground truth*.
- **Usuarios:** incluí las 104 cuentas comprometidas más una muestra aleatoria de 4 000 usuarios normales, estratificada por cuartil de actividad y con semilla fija, para no sesgar la línea base hacia cuentas hiperactivas ni casi inactivas.
- **Exclusión de cuentas no humanas:** excluí del universo muestreable las cuentas de máquina (terminadas en \$) y de sistema (SYSTEM, Local Service, entre otras), pues el scoring debe recaer sobre personas.

El resultado fue una muestra de 37 580 871 eventos para 4 104 usuarios, que persistí en formato Parquet (aproximadamente 209 MB) como el conjunto de trabajo reproducible del resto del proyecto.

4.3 Calidad de los datos

Documenté las particularidades de calidad de esta fuente real. El campo `auth_type` está ausente (marcado con «?») en el 60.2% de los eventos, y `logon_type` en el 18.7%. En lugar de descartar o imputar a ciegas, traté el valor nulo como información en sí mismo, pues su patrón no es aleatorio sino que depende del mecanismo de recolección de cada evento. Asimismo, el archivo de *red team* contiene 749 líneas de las cuales 12 líneas distintas aparecen duplicadas (34 filas en exceso); las deduplico a 715 eventos únicos, correspondientes a 104 cuentas. Como verificación adicional a nivel de evento, confirmé que 702 de esos 715 eventos tienen su línea de autenticación exacta en la muestra, y que de esos 702, **701 fueron autenticaciones exitosas y solo 1 fue un fallo**. Este dato es una confirmación directa de la tesis del proyecto: el atacante con credenciales robadas no rompe puertas, las abre.

5. Análisis exploratorio de datos

El objetivo de mi análisis exploratorio fue triple: caracterizar el comportamiento normal de autenticación, cuantificar el desbalance real y descubrir empíricamente las señales de compromiso que después convertiría en variables. Adopté un criterio deliberado de presentación: pocas figuras, cada una respondiendo una pregunta concreta del problema, en lugar de saturar el documento con gráficas sin valor analítico.

5.1 El ritmo de la red

Los tiempos del conjunto son anónimos: son una época que arranca en 1, sin fecha real, por seguridad. Mi primera pregunta fue si podía recuperar el ritmo humano de la red únicamente a partir de la periodicidad de los datos. La Figura 1 muestra que sí. Al agregar los eventos de origen humano por hora relativa del día (el residuo de dividir el tiempo entre 86 400 segundos), emerge un ciclo circadiano claro con un horario laboral inferido de 7:00 a 16:00. Un matiz metodológico relevante: existe una base de automatización que corre 24/7 bajo cuentas de usuario —el valle nocturno conserva cerca del 59% de la actividad del pico— por lo que definí la frontera de «horario laboral» como el punto medio entre el valle y el pico, y no como un porcentaje del pico. La serie diaria, además, revela los fines de semana sin necesidad de un calendario: aparecen como caídas de dos días con periodicidad siete. El *red team* ataca en 18 de los 30 días.

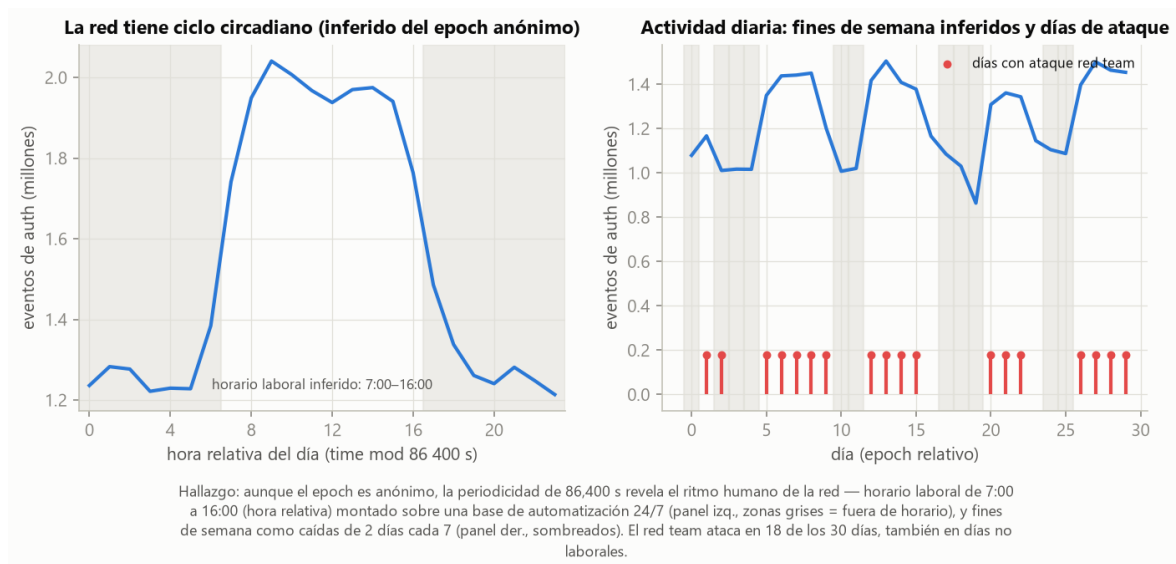


Figura 1: Ritmo de la red inferido del tiempo anónimo. Izquierda: ciclo circadiano y horario laboral inferido (zonas grises = fuera de horario). Derecha: serie diaria con fines de semana sombreados y días con ataque marcados en rojo.

5.2 Las señales del compromiso

Comparé los 181 usuario-días comprometidos contra los 47 775 normales sobre tres señales candidatas (Figura 2). El hallazgo central, y la señal más citada en la literatura sobre este conjunto, es el **alcance**: la mediana de computadoras destino distintas es de 22 en los días comprometidos frente a 10 en los normales —la huella del movimiento lateral. El segundo hallazgo fue contraintuitivo y resultó valioso: el compromiso **no** se parece a un ataque de fuerza bruta. Los días comprometidos presentan fallos con más frecuencia (50 % de los días frente a 24 %) pero en proporción mínima (un ratio típico de 0.003), porque quien usa credenciales válidas robadas avanza con éxito. De hecho, el fallo masivo pertenece a la población normal: el 17 % de los usuario-días normales son 100 % fallos, la firma de la automatización con credenciales caducas. Finalmente, el protocolo NTLM —asociado en la literatura a técnicas de movimiento lateral tipo *pass-the-hash*, catalogadas por el marco MITRE ATT&CK [11]— aparece con una mediana de ratio de 0.02 en los comprometidos frente a 0 en los normales.

5.3 El movimiento lateral hecho visible

Para darle una cara concreta al movimiento lateral, construí el grafo de conexiones usuario→computadora de una cuenta comprometida, antes y durante el ataque (Figura 3). En sus ocho días de historia previa, la cuenta acumuló 27 destinos distintos; el día del ataque tocó 49, de los cuales 28 eran nuevos —su grafo se expandió más en un solo día que en toda su historia previa— y 11 de esos destinos nuevos están confir-

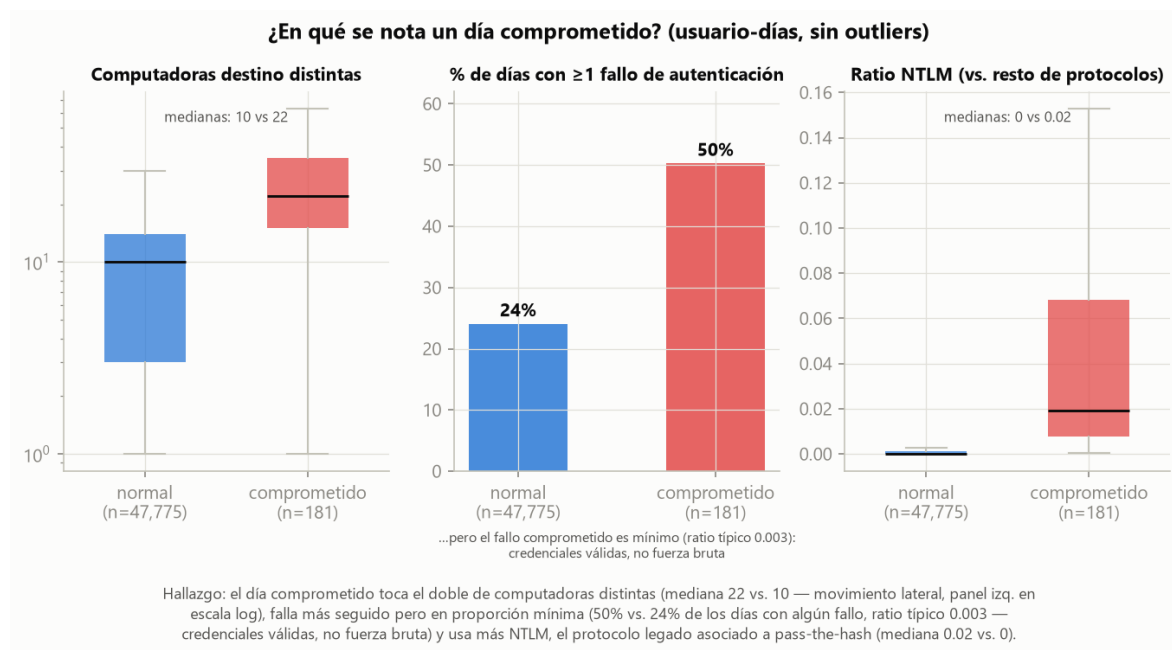


Figura 2: Comparación normal vs. comprometido sobre tres señales. El panel de destinos distintos hace visible el movimiento lateral; el de fallos revela que el compromiso usa credenciales válidas; el de NTLM confirma el protocolo legado como señal.

mados en el *ground truth*. Ninguna de esas autenticaciones falló ni disparó una alerta tradicional: la anomalía solo existe a nivel del patrón. Esta figura se convirtió en la «historia visual» del proyecto y en el prototipo del mini-grafo que después implementé en la interfaz.

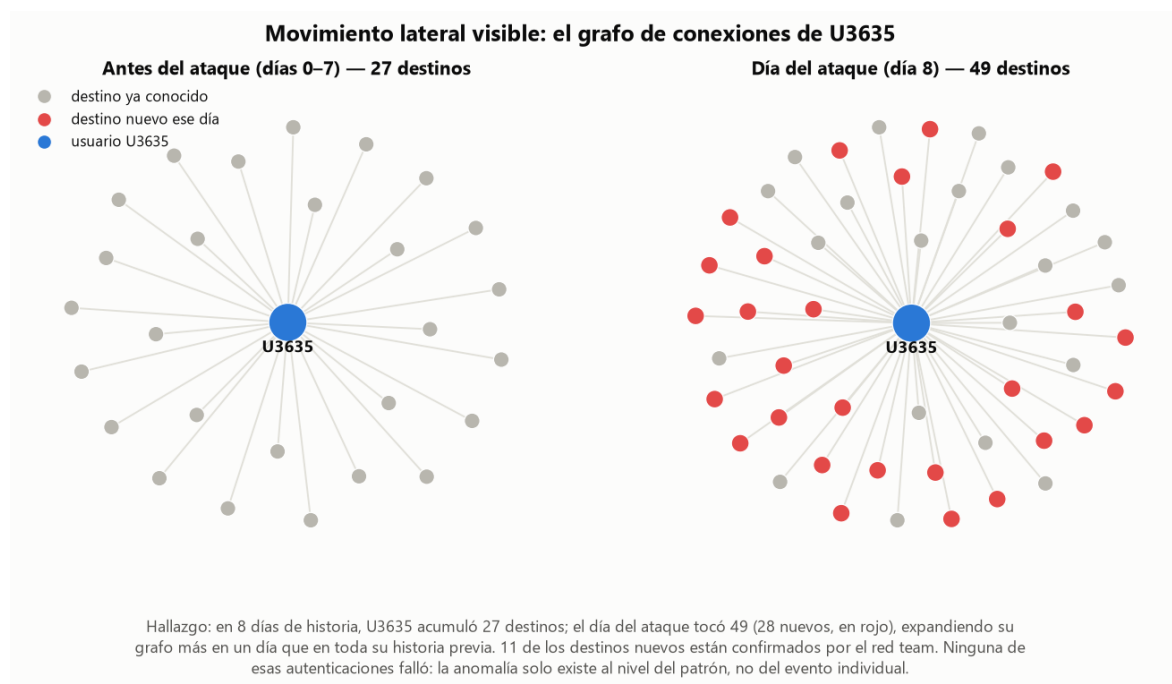


Figura 3: Grafo ego de una cuenta comprometida. En gris, los destinos que la cuenta ya conocía; en rojo, los que tocó por primera vez el día del ataque. El movimiento lateral es la novedad de conexiones.

5.4 El desbalance

No presento el desbalance como una gráfica sino como un indicador, porque un número tan extremo se comunica mejor así: de los 47 956 usuario-días de la muestra, solo 181 contienen actividad maliciosa (el 0.38 %, es decir, uno de cada 265), y esto en una muestra deliberadamente enriquecida con la totalidad de las cuentas comprometidas —la población real es del orden de 50 veces más desbalanceada. Este único dato es el argumento definitivo contra el uso de la exactitud como métrica y la justificación de todas las decisiones de evaluación que tomé después.

6. Ingeniería de variables

Considero esta la sección crítica del proyecto. Mi principio rector fue que cada variable respondiera una pregunta de desviación, y las organicé en cuatro niveles, donde cada nivel existe porque el anterior tiene un punto ciego que un atacante disciplinado podría explotar. El resultado es una tabla maestra de 47 956 usuario-días descritos por 61 variables, sin valores faltantes ni infinitos.

- **(a) Variables crudas —¿qué hizo hoy?** Capturan cada táctica concreta: el **volumen** (número de eventos, logones), el **alcance** (computadoras destino y origen distintas), los **fallos** (incluida la ráfaga máxima de fallos consecutivos, firma de fuerza bruta), el **protocolo** (conteos de NTLM, Kerberos, tickets TGT/TGS y el propio nulo como información), el **tipo de sesión** y las variables **temporales** que usan las fronteras circadianas inferidas en el EDA.
- **(b) Desviación personal —¿qué tan distinto de su propio pasado?** Es el corazón del proyecto. Para cada variable relevante calculo su z-score respecto a la media histórica del propio usuario, su razón contra el máximo histórico y banderas de «primera vez». Lo «normal» es personal: 22 destinos son rutina para un administrador y una alarma para un contador.
- **(c) Desviación grupal —¿qué tan distinto de sus pares?** Resuelve el punto ciego de los eventos globales legítimos (una actualización que dispara la actividad de todos). Como los datos están desidentificados y no hay roles organizacionales disponibles, construyo los pares *conductualmente* mediante agrupamiento (*clustering*) sobre el perfil de cada usuario, y mido la desviación del día contra la distribución de su grupo.
- **(d) Novedad de grafo —¿qué tan nuevo es su grafo de conexiones?** Cierra el punto ciego del atacante que mantiene volumen y horario normales pero que, por definición del movimiento lateral, no puede evitar tocar máquinas

que la cuenta jamás visitó. Calculo el número de aristas nuevas, su proporción, la rareza media de los destinos y el número de máquinas origen nuevas.

6.1 La regla anti-fuga

El error metodológico más probable y costoso en un proyecto con partición temporal y variables históricas es la **fuga de información** del futuro hacia el pasado. Lo traté como una prioridad de primer orden y establecí reglas estrictas: toda estadística histórica de la desviación personal se calcula desplazando la serie del usuario un día (excluyendo el día evaluado de su propia línea base); el grafo de referencia de la novedad se «conge-la» al día anterior antes de evaluar cada día; el modelo de pares se ajusta únicamente con días de entrenamiento; y ninguna variable deriva, directa o indirectamente, del archivo de etiquetas. Estas reglas las verifiqué explícitamente, línea por línea, mediante una auditoría dedicada del código.

Como ilustración del poder de este diseño, el día del ataque de la cuenta que analicé en el EDA presenta un volumen de actividad prácticamente normal (su z-score de volumen es de apenas 0.69, menos de una desviación estándar), de modo que un detector de volumen no lo vería. Pero las desviaciones de alcance y novedad lo delatan por completo: tocó destinos a 7.7 desviaciones estándar de su línea base personal y a 4.5 desviaciones de sus pares conductuales ese mismo día.

6.2 Etiquetado y partición temporal

Etiqueté cada usuario-día con $y = 1$ si contiene al menos un evento del *red team*, verificando una cobertura perfecta: los 181 pares (usuario, día) del *ground truth* tienen su fila correspondiente en la tabla maestra. La partición es **temporal estricta**, nunca aleatoria: un *k-fold* aleatorio filtraría información porque los días de un mismo usuario están autocorrelacionados y las variables históricas cruzarían el corte. Verifiqué además que ambos lados de la partición contuvieran días de ataque.

7. Modelado

Adopté un enfoque híbrido que combina un modelo no supervisado como línea base con un modelo supervisado como componente central, más un modelo de agrupamiento con doble propósito.

7.1 Los tres conjuntos

Refiné la partición temporal en tres conjuntos cronológicamente disjuntos: entrenamiento (días 0–13, con 120 positivos), validación (días 14–19, con 27 positivos) y prueba fuera de tiempo (días 20–29, con 34 positivos). El corte de validación no fue arbitrario: lo elegí con los datos, tras verificar que un corte en el día 16 dejaría la validación sin un solo positivo. Todo lo que requiere decidir —hiperparámetros, calibrador, modelo ganador y umbrales— lo decidí en validación; el conjunto de prueba lo reservé para una única evaluación final.

7.2 Modelos no supervisados

Como línea base empleé un **Isolation Forest** [4], que aísla anomalías mediante particiones aleatorias del espacio. Lo entrené sin etiquetas y solo con datos de entrenamiento, y lo evalué a posteriori contra las etiquetas. Tomé una decisión que quiero justificar explícitamente: mantuve el parámetro de contaminación en su modo automático en lugar de fijarlo con la tasa real de ataque, porque fijarlo con esa tasa sería filtrar información de las etiquetas a un modelo que se presume no supervisado. Su desempeño fue el esperado para una línea base: un PR-AUC de 0.027, útil como piso de referencia pero insuficiente como detector único.

El segundo modelo no supervisado fue un **K-Means** [7] sobre el perfil de cada usuario, con el número de grupos elegido por el coeficiente de silueta [8]. Este modelo cumple un doble propósito: genera las variables de desviación grupal del nivel (c) y, como puntuación de anomalía por distancia al centroide del rol del usuario, alcanza un PR-AUC de 0.093, superando a la línea base de Isolation Forest con un modelo mucho más simple.

7.3 Segundo conflicto: el agrupamiento degenerado

La primera ejecución del K-Means produjo una solución degenerada que documenté como un conflicto real: el criterio de silueta seleccionó $k = 2$ con un grupo de **un solo usuario** y una silueta engañosamente alta de 0.997. Una cuenta atípica extrema secuestraba el agrupamiento, y un grupo unipersonal no es un grupo de pares (no hay nadie contra quién compararse). Lo corregí con dos salvaguardas: una transformación logarítmica sobre las variables de volumen antes del escalado, y la regla de que un k solo es candidato si su grupo más pequeño agrupa al menos el 1 % de los usuarios. Con ello obtuve una solución sana de $k = 2$ (silueta 0.63) con dos roles conductuales interpretables: «cuentas ligeras e intermitentes» y «cuentas pesadas siempre encendidas». Documento este episodio porque es exactamente el tipo de decisión que separa

un agrupamiento decorativo de uno útil.

7.4 Modelo supervisado central

El modelo central es **XGBoost** [5], una implementación escalable de *gradient boosting* sobre árboles. Manejé el desbalance con el parámetro `scale_pos_weight` fijado en 184.9 —el cociente de negativos sobre positivos del conjunto de entrenamiento, no de la tabla completa. El número de árboles no lo fijé como hiperparámetro sino que resultó de una detención temprana por el PR-AUC de validación (el modelo ganador se detuvo en 62 árboles). Realicé una búsqueda aleatoria de 40 configuraciones sobre siete hiperparámetros, evaluadas sobre la validación temporal y nunca sobre un *k-fold* aleatorio. La configuración ganadora es deliberadamente pequeña y regularizada (profundidad máxima 3, tasa de aprendizaje $\approx 0,10$, peso mínimo por hoja ≈ 17), lo cual tiene todo el sentido con solo 120 positivos de entrenamiento. Como referencias comparativas entrené también un Random Forest y una Regresión Logística.

7.5 Iteración de variables

Tras el primer modelo cumplí la iteración obligatoria sobre la ingeniería de variables: probé agregar la distancia al centroide del K-Means como una variable adicional del modelo supervisado. La **rechacé** con base en los datos, pues el PR-AUC de validación fue ligeramente inferior con ella (0.2227) que sin ella (0.2264): la información que aportaba ya estaba capturada por las variables de desviación grupal. Documento la prueba y la decisión negativa porque un «no mejoró, no se queda» respaldado por números vale más que acumular variables por inercia.

7.6 Selección del mejor modelo y calibración

Seleccioné el modelo en validación, con el PR-AUC como criterio rector. XGBoost ganó por más del doble sobre las referencias, como muestra la Tabla 1. La ventaja del *boosting* con re-pesado del gradiente sobre el promediado de árboles del Random Forest, y sobre la incapacidad de la regresión lineal para capturar interacciones (como «NTLM alto y destinos nuevos»), quedó cuantificada.

La salida cruda de un XGBoost re-pesado no es una probabilidad interpretable, sino un valor inflado por diseño. Como la puntuación de riesgo final y los umbrales del SOC se construyen sobre ella, la **calibré** —ajustando el calibrador únicamente con validación. No elegí el método por dogma: comparé la calibración isotónica contra la de Platt mediante el *Brier score* en validación cruzada interna, y ganó la isotónica

Modelo	PR-AUC (val)	Recall @ FPR = 1 % (val)
XGBoost (central)	0.226	0.63
Random Forest	0.107	0.37
Regresión Logística	0.085	0.33

Cuadro 1: Selección del mejor modelo en el conjunto de validación. XGBoost domina bajo la métrica rectora y bajo el criterio de desempate.

por un margen mínimo (0.00294 frente a 0.00299), lo que respaldó empíricamente la elección.

7.7 Evaluación final fuera de tiempo

Con todas las decisiones ya tomadas, evalué una única vez sobre el conjunto de prueba fuera de tiempo. La Tabla 2 y la Figura 4 resumen los resultados. El PR-AUC de prueba fue de 0.056: una caída real respecto al 0.226 de validación, cuyas causas son identificables —solo 34 positivos en prueba y campañas de ataque distintas a las de entrenamiento. Reporto también el ROC-AUC de 0.939, pero con la advertencia explícita de que, con un 0.19 % de positivos, un ROC-AUC alto es engañosamente cómodo: el 99.8 % de los pares que compara son triviales. La curva de precisión-recall del panel izquierdo cuenta la historia verdadera.

Métrica (prueba fuera de tiempo)	Modelo calibrado	Isolation Forest
PR-AUC	0.056	0.027
ROC-AUC (con advertencia)	0.939	0.759
Recall @ FPR = 1 %	0.34	0.12
Recall @ FPR = 0.1 %	0.10	0.09

Cuadro 2: Evaluación final sobre el conjunto de prueba fuera de tiempo (días 20–29). La tasa base de positivos es del 0.19 %.

El panel derecho de la Figura 4 es, en mi opinión, la herramienta más valiosa para el negocio, pues traduce el desempeño estadístico en lenguaje operativo: con un presupuesto de 20 alertas diarias, el sistema cubre el 31 % de las cuentas comprometidas del periodo; con 100 alertas diarias, el 85 %. Esta curva convierte la pregunta «¿qué tan bueno es el modelo?» en «¿cuántos analistas cuesta cada punto de recall?».

7.8 Reporte de estabilidad

Un modelo que discrimina pero se desestabiliza entre periodos no es desplegable. Por ello incluí un reporte de estabilidad sobre el periodo fuera de tiempo, con los indicadores estándar de la industria del riesgo [10]. El **PSI** (*Population Stability Index*) de la

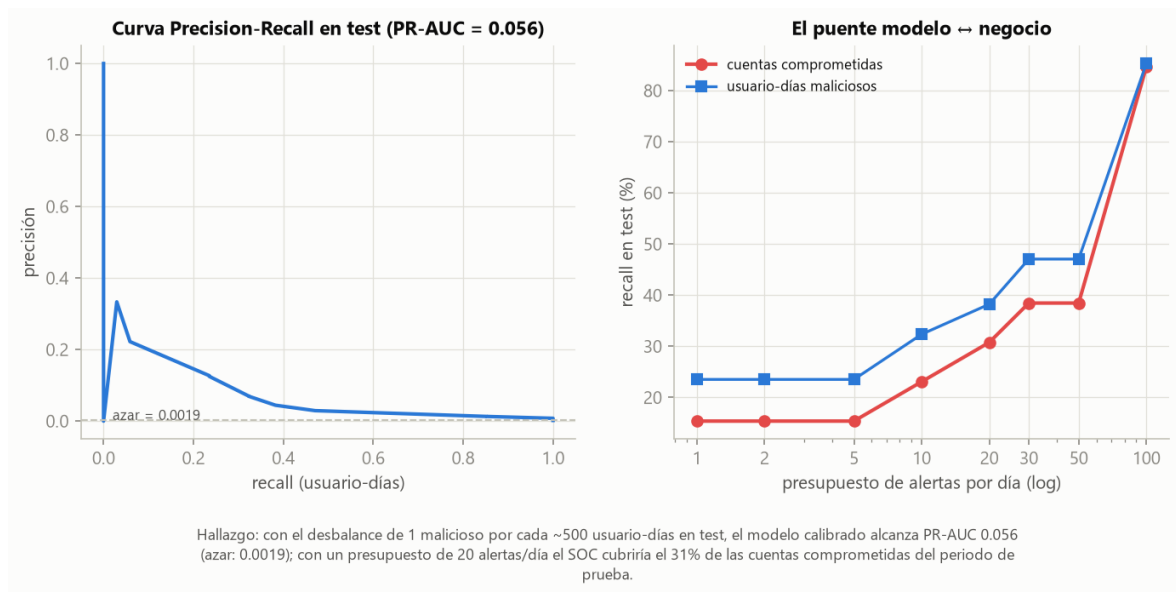


Figura 4: Izquierda: curva precisión-recall en prueba (la línea punteada es el azar). Derecha: el puente entre el modelo y el negocio —recall de cuentas comprometidas en función del presupuesto de alertas por día.

puntuación entre entrenamiento y prueba fue de 0.019, dentro del rango «estable»: la distribución de scores que vería el SOC apenas se movió entre periodos. El estadístico **KS** de clasificación fue de 0.74, indicando una separación fuerte entre las poblaciones de positivos y negativos. El **CSI** (*Characteristic Stability Index*) de las variables reveló que dos de ellas —las desviaciones de fallos y de NTLM respecto a los pares— sí derivan entre periodos, lo cual no es un defecto del *pipeline* sino un hecho del fenómeno: ambas dependen de *cómo* ataca el *red team* en cada campaña, que cambia. El resto de las variables, incluidas todas las de grafo, permanecen estables, y el *score* agregado se mantiene estable pese a la deriva de esas dos entradas.

7.9 Explicabilidad

Cumpliendo el principio de diseño de que ninguna puntuación viaje sin su explicación, empleé valores SHAP [6] para interpretar el modelo. El resumen global (Figura 5) confirma que las señales dominantes son el ratio de NTLM, el número de máquinas origen nuevas y la rareza media de los destinos —es decir, protocolo anómalo y novedad de grafo, no volumen crudo. El modelo aprendió la tesis del proyecto, no un atajo. Esta misma técnica, en su versión local por usuario-día, es la que alimenta la explicación que el analista ve en la interfaz.

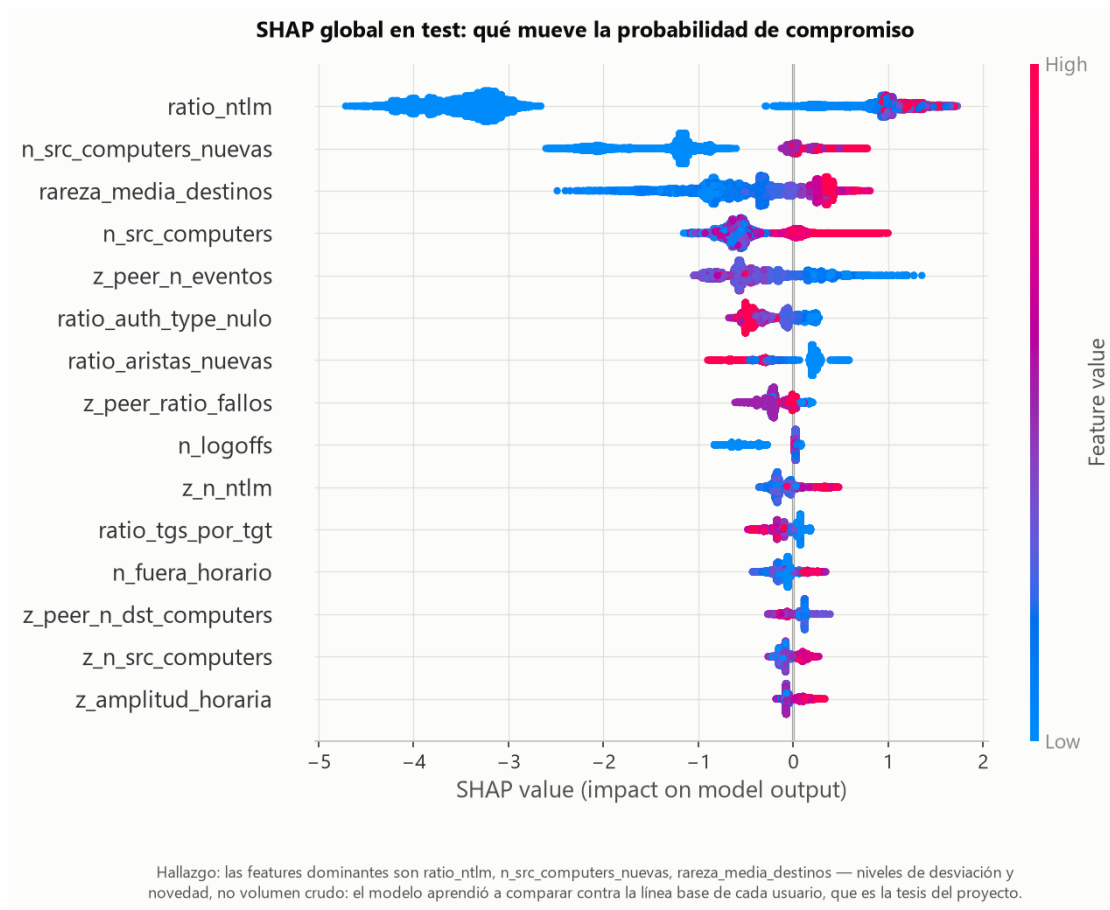


Figura 5: Resumen global de valores SHAP sobre el conjunto de prueba. Dominan las variables de protocolo y de novedad de conexiones, validando el diseño multinivel de la ingeniería de variables.

8. Análisis crítico y conflictos del desarrollo

Considero que la honestidad sobre las limitaciones es parte del rigor científico, por lo que dedico una sección al análisis crítico de los resultados y a los conflictos que enfrenté, más allá de los ya narrados en la adquisición de datos y el agrupamiento degenerado.

8.1 Tercer conflicto: una autocorrección metodológica

Antes de aceptar los resultados como definitivos, exploré tres palancas para mejorarlos, y durante esa exploración cometí —y corregí— un error metodológico que documento precisamente porque su corrección es evidencia de rigor. Al probar la agregación del riesgo a nivel de cuenta (tomar el máximo de los últimos K días), inicialmente elegí $K = 7$ porque era el valor que mejor se veía *en el conjunto de prueba*, obteniendo un PR-AUC aparentemente superior de 0.084. Esto es exactamente la fuga metodológica que había estado combatiendo durante todo el proyecto: elegir un hiperparámetro mirando el conjunto de prueba. Lo detecté, rehíce la selección eligiendo K correctamente sobre

la validación —que resultó en $K = 1$, es decir, sin agregación— y el «resultado» de mejora se desvaneció.

8.2 La exploración honesta de mejoras

Probé tres palancas y ninguna sobrevivió a una selección honesta: (1) extender la ventana temporal no añade positivos, pues verifiqué que no existen ataques fuera de los días 1–29 en el conjunto completo; (2) combinar las puntuaciones de XGBoost y K-Means no generaliza, pues el peso óptimo en validación no transfiere a prueba; y (3) la agregación por cuenta, correctamente evaluada, no mejora. Esto no es un fracaso del *pipeline*: con solo 181 eventos de compromiso en todo el conjunto, no hay margen estadístico para que el ajuste fino encuentre algo que generalice, y el techo de esta fuente de datos con este enfoque ya se alcanzó.

8.3 Contextualización honesta de los resultados

Los resultados son modestos en términos absolutos pero defendibles como primera iteración de un problema genuinamente difícil. Un análisis de *bootstrap* sobre los 34 positivos de prueba arroja un intervalo de confianza del 95% para el PR-AUC de $[0,022, 0,146]$: amplio por el tamaño de muestra, pero cuyo límite inferior sigue estando cerca de nueve veces por encima de un ordenamiento aleatorio (cuyo PR-AUC medio medí en 0.0025). La afirmación «el modelo discrimina mejor que el azar» es, por tanto, sólida y no frágil. La honestidad exige reconocer también el costo: al umbral operativo naranja, la precisión es baja (13 verdaderos positivos frente a 280 falsos positivos), lo que refleja el reto real de este desbalance. El punto de comparación correcto para juzgar el modelo no es un umbral absoluto de «bueno», sino el *lift* sobre el azar y la estabilidad del ordenamiento, criterios bajo los cuales el sistema cumple.

9. Resolución del problema

La utilidad de un modelo se materializa únicamente cuando es accesible para su usuario final. Dedicué la última parte del proyecto a convertir los modelos entrenados en un sistema operativo completo.

9.1 Pipeline de inferencia y puntuación de riesgo

Serialicé todos los artefactos —el XGBoost calibrado, el Isolation Forest, el K-Means, los umbrales y el explicador SHAP— y construí el flujo de inferencia que convierte

un usuario-día en una decisión accionable. La puntuación de riesgo (0–100) sigue la fórmula híbrida $\text{riesgo} = 100 \cdot (w \cdot p_{\text{sup}} + (1 - w) \cdot a_{\text{norm}})$, donde p_{sup} es la probabilidad calibrada y a_{norm} la anomalía normalizada. En lugar de adoptar un peso w arbitrario, lo elegí en validación por PR-AUC, y el resultado ($w = 0,95$) confirma con transparencia que el componente supervisado domina. Los niveles del semáforo los recalibré en el espacio 0–100 según la capacidad operativa del SOC.

9.2 Consumo de modelos: la API

Expuse los modelos mediante una API construida con FastAPI, que sirve una instantánea precomputada del periodo de prueba a través de ocho *endpoints* tipados: los indicadores generales, la cola de triaje priorizada, la serie temporal de riesgo por usuario, la explicación SHAP con comparativas, el resumen de actividad, el registro de decisiones del analista y un *endpoint* de inferencia en vivo. La API es la dueña de todo el conocimiento del modelo; el contrato queda documentado automáticamente mediante OpenAPI, y validé su comportamiento con una suite de 23 pruebas de contrato.

9.3 La interfaz de usuario

Desarrollé la interfaz con React (mediante Vite y TypeScript), desplegada en producción, con tres vistas que atienden a los tres perfiles de usuario: una **consola de triaje** con la cola priorizada, la evolución del riesgo y la explicación SHAP; una vista de **investigación** que incluye el mini-grafo ego que hace visible el movimiento lateral y el historial de decisiones; y un **panel ejecutivo** con la tendencia de riesgo por rol conductual y la eficiencia del SOC. Un principio gobernó todo el frontend: la interfaz nunca recalcula el riesgo; si un número no proviene de la API, no existe.

9.4 Asistente conversacional

Como componente adicional de usabilidad, integré un asistente conversacional que responde preguntas del usuario en lenguaje natural y sirve de guía. Lo implementé con la técnica de *function calling*: el modelo de lenguaje, cuando necesita un dato, invoca herramientas que consultan la instantánea real del modelo, de modo que responde con las cifras verdaderas del periodo en lugar de inventarlas. Por seguridad, la clave de la API del proveedor reside exclusivamente en el servidor y nunca se expone al navegador.

10. Conclusiones

Este proyecto me permitió recorrer el ciclo completo de un problema de ciencia de datos aplicado a un dominio real y difícil, desde mil millones de eventos crudos hasta una interfaz funcional en producción. Extraigo varias conclusiones.

En lo metodológico, confirmé que la elección del aprendizaje supervisado está sólidamente justificada para este problema: dado que dispongo de etiquetas verificadas del *red team*, puedo optimizar directamente la función objetivo del negocio y medir el desempeño de forma objetiva, algo que un enfoque puramente no supervisado —que solo identificaría desviaciones estadísticas sin distinguir fraude genuino de comportamiento legítimo atípico— no permitiría. No obstante, integré ambos paradigmas: el modelo no supervisado aporta la línea base y, vía el agrupamiento de pares, genera las variables de desviación grupal que resultaron valiosas.

En lo técnico, el hallazgo que considero más relevante es que la señal del uso indebido de credenciales vive en la **desviación** y en la **novedad de conexiones**, no en el volumen crudo, tal como confirmó el análisis SHAP. La ingeniería de variables en cuatro niveles —y en particular la regla anti-fuga que la sostiene— es, en mi valoración, la contribución técnica central del trabajo.

En cuanto a los resultados, fui deliberadamente honesto: son modestos en términos absolutos (un PR-AUC de prueba de 0.056) pero representan una discriminación real y estable, muy por encima del azar, sobre un problema con solo 181 eventos de compromiso y un desbalance extremo. La curva de alertas por día frente a recall demuestra que, aun con estas métricas, el sistema aporta valor operativo: reduce el pajar a una lista corta y priorizada. La utilidad plena se consolidaría con la línea de trabajo futuro más prometedora —y la única que añadiría señal genuina en lugar de reordenar la existente— que es el enriquecimiento con fuentes de datos adicionales del propio conjunto de LANL (procesos y flujos de red), contempladas como opcionales en el diseño original.

Finalmente, valoro que los conflictos que enfrenté —el *data fence* en la adquisición, el agrupamiento degenerado y, sobre todo, la autocorrección de la fuga metodológica— no fueron obstáculos que ocultar, sino oportunidades para demostrar el rigor con el que abordé el proyecto. La forma en que un sistema falla y se corrige dice tanto de su calidad como sus métricas.

11. Referencias

- [1] Verizon. (2024). *2024 Data Breach Investigations Report (DBIR)*. Verizon Business. <https://www.verizon.com/business/resources/reports/dbir/>
- [2] Kent, A. D. (2015). *Comprehensive, Multi-Source Cyber-Security Events* [Data set]. Los Alamos National Laboratory. <https://csr.lanl.gov/data/cyber1/>
- [3] Kent, A. D. (2015). Cybersecurity Data Sources for Dynamic Network Research. En *Dynamic Networks and Cyber-Security*. Imperial College Press.
- [4] Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation Forest. *Proceedings of the 2008 Eighth IEEE International Conference on Data Mining (ICDM)*, 413–422.
- [5] Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.
- [6] Lundberg, S. M., & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. *Advances in Neural Information Processing Systems (NeurIPS)*, 30, 4765–4774.
- [7] MacQueen, J. (1967). Some Methods for Classification and Analysis of Multivariate Observations. *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, 1, 281–297.
- [8] Rousseeuw, P. J. (1987). Silhouettes: A Graphical Aid to the Interpretation and Validation of Cluster Analysis. *Journal of Computational and Applied Mathematics*, 20, 53–65.
- [9] Saito, T., & Rehmsmeier, M. (2015). The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets. *PLoS ONE*, 10(3), e0118432.
- [10] Siddiqi, N. (2006). *Credit Risk Scorecards: Developing and Implementing Intelligent Credit Scoring*. John Wiley & Sons. (Referencia para los índices de estabilidad PSI, CSI y el estadístico KS.)
- [11] The MITRE Corporation. (2023). *MITRE ATT&CK: Lateral Movement (TA0008) y Use Alternate Authentication Material: Pass the Hash (T1550.002)*. <https://attack.mitre.org/>